

Dialog Personalizado

Un diálogo es una ventana pequeña que le indica al usuario que debe tomar una decisión o ingresar una información adicional. Un diálogo no ocupa toda la pantalla y generalmente se usa para eventos modales que requieren que los usuarios realicen alguna acción para poder continuar.

Se crea un nuevo proyecto de nombre `EjemploDialog` con un `Empty Views Activity` que tendrá un texto y un botón en `activity_main.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Centro de Estudios Almi Ikastetxea"
        android:textSize="20sp" />
    <Button
        android:id="@+id/btnAcceder"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Acceder" />
</LinearLayout>
```

En la clase `MainActivity.java` se implementa el click del botón de acceder y por el momento se dejará vacío:

```
package com.example.ejemplodialog;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
```

```
public class MainActivity extends AppCompatActivity
{
    private Button btnAcceder;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.ma
in), (v, insets) ->
        {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom);
            return insets;
        });

        btnAcceder = findViewById(R.id.btnAcceder);
        btnAcceder.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)
            {
                // Pendiente: mostrar el diálogo de login (se
completa más adelante)
            }
        });
    }
}
```

Una vez capturado el click se crea el diseño para el Dialog, para el que se necesitará el logo pegado en la carpeta Drawable. Una vez copiado se crea un Layout de nombre dialog_personalizado.xml con 2 campos de texto, dos etiquetas y dos botones que solicitarán usuario y contraseña. Además habrá que integrar el logo:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="30dp">

    <ImageView
        android:layout_width="match_parent"
        android:layout_height="80dp"
        android:layout_gravity="center"
        android:src="@drawable/logo_almi" />
```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Usuario"
        android:textSize="20sp" />

    <EditText
        android:id="@+id/etUser"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:autofillHints="username"
        android:inputType="text" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Password"
        android:textSize="20sp" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:autofillHints="password"
        android:inputType="textPassword" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btnAceptar"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Aceptar" />

    <Button

```

```

        android:id="@+id/btnCancelar"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Cancelar" />
    </LinearLayout>
</LinearLayout>

```

Se crea una nueva clase de nombre `LoginDialogFrag` en el mismo package que `MainActivity.java` en la que se implementarán los botones del Dialog. Se implementarán varios eventos entre los que destacan `OnCreateView`, que es donde se recogerán las instancias de los componentes gráficos:

```

package com.example.ejemplodialog;

import android.app.Dialog;
import android.content.DialogInterface;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.DialogFragment;

public class LoginDialogFrag extends DialogFragment
{
    private EditText etNombre;
    private EditText etPassword;
    private Button btnAceptar;
    private Button btnCancelar;

    public LoginDialogFrag()
    {
        super();
    }

    @Override
    public void onCancel(@NonNull DialogInterface dialog)
    {
        super.onCancel(dialog);
    }

    // Cierre del diálogo
    @Override
    public void onDetach()
    {
        super.onDetach();
    }
}

```



```

        @NonNull
        @Override
        public Dialog onCreateDialog(@Nullable Bundle savedInstanceState)
        {
            Dialog dialog =
super.onCreateDialog(savedInstanceState);
            dialog.setTitle("Login");
            return dialog;
        }

        // Crear la vista (instancias de los componentes)
        @Nullable
        @Override
        public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container, @Nullable Bundle savedInstanceState)
        {
            View vista =
inflater.inflate(R.layout.dialog_personalizado, container,
false);

            etNombre = vista.findViewById(R.id.etUser);
            etPassword = vista.findViewById(R.id.etPassword);
            btnAceptar = vista.findViewById(R.id.btnAceptar);
            btnCancelar = vista.findViewById(R.id.btnCancelar);

            return vista;
        }

        // Se ejecuta tras crear la vista (aquí irán los clicks de
los botones)
        @Override
        public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState)
        {
            super.onViewCreated(view, savedInstanceState);
        }
    }
}

```

En MainActivity.java se implementa el click que se dejó vacío con anterioridad, instanciando y mostrando el Dialog creado anteriormente:

```

package com.example.ejemplodialog;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;

```

```
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity
{
    private Button btnAcceder;
    private LoginDialogFrag dialog;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.ma
in), (v, insets) ->
        {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom);
            return insets;
        });

        btnAcceder = findViewById(R.id.btnAcceder);
        btnAcceder.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)
```

```

        {
            dialog = new LoginDialogFrag();
            dialog.show(getSupportFragmentManager(),
"login");
        }
    });
}
}

```

A continuación en LoginDialogFrag se programan los clicks de los botones de aceptar y cancelar del cuadro de diálogo:

```

package com.example.ejemplodialog;

import android.app.Dialog;
import android.content.DialogInterface;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.DialogFragment;

public class LoginDialogFrag extends DialogFragment
{
    private EditText etNombre;
    private EditText etPassword;
    private Button btnAceptar;
    private Button btnCancel;

    public LoginDialogFrag()
    {
        super();
    }

    @Override
    public void onCancel(@NonNull DialogInterface dialog)
    {
        super.onCancel(dialog);
    }

    // Cierre del diálogo
    @Override
    public void onDetach()

```

```

    {
        super.onDetach();
    }

    @NonNull
    @Override
    public Dialog onCreateDialog(@Nullable Bundle savedInstanceState)
    {
        Dialog dialog =
super.onCreateDialog(savedInstanceState);
        dialog.setTitle("Login");
        return dialog;
    }

    // Crear la vista (instancias de los componentes)
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container, @Nullable Bundle savedInstanceState)
    {
        View vista =
inflater.inflate(R.layout.dialog_personalizado, container,
false);

        etNombre = vista.findViewById(R.id.etUser);
        etPassword = vista.findViewById(R.id.etPassword);
        btnAceptar = vista.findViewById(R.id.btnAceptar);
        btnCancelar = vista.findViewById(R.id.btnCancelar);

        return vista;
    }

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState)
    {
        super.onViewCreated(view, savedInstanceState);

        btnCancelar.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)
            {
                dismiss();
            }
        });

        btnAceptar.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)

```

```

        {
            String nombre = etNombre.getText().toString();
            String password =
etPassword.getText().toString();

            if (nombre.equals("Almi") &&
password.equals("Almi123"))
            {
                Toast.makeText(getContext(), "Bienvenido",
Toast.LENGTH_SHORT).show();
            }
            else
            {
                dismiss();
            }
        }
    });
}
}

```

En siguiente lugar se comenzará con la implementación del uso de SQLite, que es una base de datos que permite a Android almacenar información estructurada de forma local. Para ello se crea una nueva actividad de nombre CentralActivity con el siguiente diseño:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
tools:context=".CentralActivity">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="DAM ALMI"
        android:gravity="center"
        android:textSize="40sp" />

    <Button
        android:id="@+id/btnSensores"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Sensores" />

    <Button
        android:id="@+id/btnVoz"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="VOZ" />

    <Button

```



```

        android:id="@+id/btncamara"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Camara" />

        <Button
            android:id="@+id/btnvolley"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Volley" />

    </LinearLayout>

```

A continuación habrá que modificar el Dialog para que se lance la nueva actividad al introducir las credenciales correctas:

```

package com.example.ejemplodialog;

import android.app.Dialog;
import android.content.DialogInterface;
import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.DialogFragment;

public class LoginDialogFrag extends DialogFragment
{
    private EditText etNombre;
    private EditText etPassword;
    private Button btnAceptar;
    private Button btnCancelar;

    public LoginDialogFrag()
    {
        super();
    }

    @Override
    public void onCancel(@NonNull DialogInterface dialog)
    {
        super.onCancel(dialog);
    }

    // Cierre del diálogo
    @Override
    public void onDetach()

```

```

    {
        super.onDetach();
    }

    @NonNull
    @Override
    public Dialog onCreateDialog(@Nullable Bundle savedInstanceState)
    {
        Dialog dialog =
super.onCreateDialog(savedInstanceState);
        dialog.setTitle("Login");
        return dialog;
    }

    // Crear la vista (instancias de los componentes)
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container, @Nullable Bundle savedInstanceState)
    {
        View vista =
inflater.inflate(R.layout.dialog_personalizado, container,
false);

        etNombre = vista.findViewById(R.id.etUser);
        etPassword = vista.findViewById(R.id.etPassword);
        btnAceptar = vista.findViewById(R.id.btnAceptar);
        btnCancelar = vista.findViewById(R.id.btnCancelar);

        return vista;
    }

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState)
    {
        super.onViewCreated(view, savedInstanceState);

        btnCancelar.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)
            {
                dismiss();
            }
        });

        btnAceptar.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)

```

```

        {
            String nombre = etNombre.getText().toString();
            String password =
etPassword.getText().toString();

            if (nombre.equals("Almi") &&
password.equals("Almi123"))
            {
                Toast.makeText(getContext(), "Bienvenido",
Toast.LENGTH_SHORT).show();
                Intent intent = new Intent(getContext(),
CentralActivity.class);
                startActivity(intent);
            }
            else
            {
                dismiss();
            }
        }
    });
}
}

```

El siguiente paso será el de crear una BBDD a nivel local que contendrá el listado de usuarios, para así validar los usuarios con la BBDD. Se creará una nueva actividad de nombre RegisterActivity para registrarse y ver los usuarios:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="5dp"
tools:context=".RegisterActivity">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Registro"
        android:gravity="center"
        android:textSize="20sp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="2"

```

```

        android:text="Usuario"
        android:gravity="center" />

<EditText
    android:id="@+id/etRegistroUser"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:autofillHints="username"
    android:inputType="text" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:text="Password"
        android:gravity="center" />

    <EditText
        android:id="@+id/etRegistroPassword"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:autofillHints="password"
        android:inputType="textPassword" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:text="Re-Password"
        android:gravity="center" />

    <EditText
        android:id="@+id/etRegistroRePassword"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:autofillHints="password"
        android:inputType="textPassword" />
</LinearLayout>

<LinearLayout

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnNuevoUsuario"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Nuevo" />

        <Button
            android:id="@+id/btnUpdateUsuario"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Actualizar" />
    </LinearLayout>

    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_marginTop="8dp"
        android:layout_marginBottom="8dp"
        android:background="?android:attr/listDivider" />

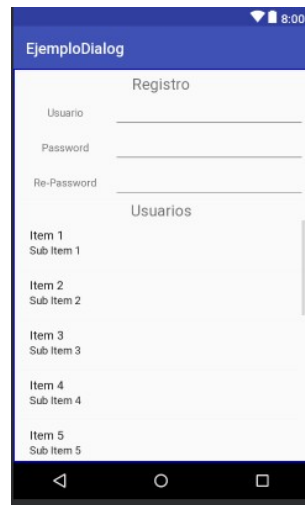
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Usuarios"
        android:gravity="center"
        android:textSize="20sp" />

    <ListView
        android:id="@+id/lvUsers"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</LinearLayout>

```

Quedando la interfaz con este aspecto:



Se agrega un botón de registro en MainActivity:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Centro de Estudios Almi Ikastetxea"
        android:textSize="20sp" />

    <Button
        android:id="@+id/btnAcceder"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Acceder" />

    <Button
        android:id="@+id/btnRegistro"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Registrar" />

</LinearLayout>
```


Y se implementa el botón en MainActivity.java:

```
package com.example.ejemplodialog;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity
{
    private Button btnAcceder;
    private Button btnRegistrar;
    private LoginDialogFrag dialog;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.ma
in), (v, insets) ->
        {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom);
            return insets;
        });

        btnAcceder = findViewById(R.id.btnAcceder);
        btnAcceder.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)
            {
                dialog = new LoginDialogFrag();
                dialog.show(getSupportFragmentManager(),
"login");
            }
        });

        btnRegistrar = findViewById(R.id.btnRegistro);
        btnRegistrar.setOnClickListener(new
View.OnClickListener()
        {
```

```

        @Override
        public void onClick(View v)
        {
            Intent intent = new
Intent(getApplicationContext(), RegisterActivity.class);
            startActivity(intent);
        }
    });
}
}

```

Para comenzar a gestionar la base de datos habrá que agregar librerías a las dependencias en el `build.gradle.kts` (Module :app) en el bloque `dependencies{...}`:

```

implementation("androidx.room:room-runtime:2.6.1")
annotationProcessor("androidx.room:room-compiler:2.6.1")

```

Se crea un nuevo package de nombre `model` y dentro se crea una clase de nombre `Usuario`, que representará el modelo de datos:

```

@Entity(tableName = "Usuario")
public class Usuario
{
    @PrimaryKey(autoGenerate = true)
    private int id;
    private String usuario;
    private String password;

    @Ignore
    public Usuario(String usuario, String password)
    {
        this.usuario = usuario;
        this.password = password;
    }

    public Usuario()
    {
    }

    public int getId()
    {
        return id;
    }

    public void setId(int id)
    {
        this.id = id;
    }
}

```

```
public String getUsuario()
{
    return usuario;
}

public void setUsuario(String usuario)
{
    this.usuario = usuario;
}

public String getPassword()
{
    return password;
}

public void setPassword(String password)
{
    this.password = password;
}
}
```

Para que el ORM de Android interprete bien el modelo habrá que comentar distintos aspectos de la creación usando la @:

- `@Entity`: da nombre a la entidad y por tanto a la tabla que contendrá los datos.
- `@PrimaryKey`: fija el nombre de la clave primaria, que en este caso será autoincremental.
- `@Ignore`: se utiliza para que el ORM no utilice ese constructor (le falta un parámetro) en la generación de la tabla.

Lo siguiente será crear un paquete de nombre `bbdd` y dentro generar una interfaz para que la clase `AppDatabase` (que se programará en el siguiente paso) pueda ejecutar queries sobre la base de datos:

```
package com.example.ejemplodialog.bbdd;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.Query;
import androidx.room.Update;
```

```
import com.example.ejemplodialog.model.Usuario;

import java.util.List;

@Dao
public interface UsuariosDao
{
    @Query("SELECT * FROM Usuario ORDER BY id")
    LiveData<List<Usuario>> loadAllUsuarios();

    @Insert
    void insertUsuario(Usuario usuario);

    @Update
    void updateUsuario(Usuario usuario);

    @Delete
    void delete(Usuario usuario);

    @Query("SELECT * FROM Usuario WHERE id = :id")
    Usuario loadUsuarioById(int id);
}
```

Dentro del package `bbdd` se crea una clase de nombre `AppDatabase` que va a gestionar la creación de la base de datos:

```
package com.example.ejemplodialog.bbdd;

import android.content.Context;
import android.util.Log;

import androidx.room.Database;
```

```
import androidx.room.Room;

import androidx.room.RoomDatabase;

import com.example.ejemplodialog.model.Usuario;

@Database(entities = {Usuario.class}, version = 1,
exportSchema = false)

public abstract class AppDatabase extends RoomDatabase
{
    private static final String LOG_TAG =
AppDatabase.class.getSimpleName();

    private static final Object LOCK = new Object();

    private static final String DATABASE_NAME = "UsuariosBD";

    private static AppDatabase sInstance;

    public static AppDatabase getInstance(Context context)
    {
        if (sInstance == null)
        {
            synchronized (LOCK)
            {
                Log.d(LOG_TAG, "Creando la base de datos");
                sInstance = Room.databaseBuilder(

context.getApplicationContext(),

AppDatabase.class,

AppDatabase.DATABASE_NAME)

                .build();

            }

        }

        return sInstance;
    }
}
```

```
public abstract UsuariosDao usuariosDao();  
}
```

Esta clase es abstracta, lo que implica que no se puede instanciar si no es a través de un método factoría. Gracias a ello se garantiza que únicamente se crea una instancia de esta clase en todo el programa, implementando de esta forma el patrón Singleton. Con la etiqueta `@Database` se fija la clase que implementa la entidad. Se fijará que no se quiere exportar el Schema porque no se desea almacenar un histórico de operaciones de la base de datos. Se crean varios atributos:

- `LOG_TAG`: que se usará para debuggear la aplicación.
- `LOCK`: que se utilizará como semáforo. Gracias a esto se podrá garantizar que no hay acceso simultáneo a la creación de la base de datos.
- `DATABASE_NAME`: nombre de la base de datos.
- `sInstance`: instancia de la base de datos, de la que se garantizará que solo exista una.

Se genera el método `getInstance`, un método factoría. El interés del mismo es que al no haber un constructor, para obtener una instancia de la base de datos hay que pasar por este método. En él se comprueba si la instancia ha sucedido ya, y en caso contrario se accede a un bloque de código marcado como `synchronized`. Esto implica que en caso de que un hilo haya accedido ya al semáforo implementado en la variable `LOCK` el resto quedarán encolados hasta que dicho hilo salga del bloque. De este modo se garantiza a nivel de procesador que no es posible la creación de una instancia en el mismo exacto momento por parte de dos hilos de ejecución distintos.

En el mismo paquete se crea una clase de nombre `AppExecutors` para poder ejecutar transacciones sobre la base de datos en distintos hilos de ejecución, es una especie de cola de hilos.

```
package com.example.ejemplodialog.bbdd;  
  
import android.os.Handler;  
import android.os.Looper;  
  
import androidx.annotation.NonNull;  
  
import java.util.concurrent.Executor;  
import java.util.concurrent.Executors;  
  
public class AppExecutors  
{
```



```
private static final Object LOCK = new Object();

private static AppExecutors sInstance;

private final Executor diskIO;

private final Executor mainThread;

private final Executor networkIO;

private static class MainThreadExecutor implements
Executor
{
    private final Handler mainThreadHandler = new
Handler(Looper.getMainLooper());

    @Override
    public void execute(@NonNull Runnable command)
    {
        mainThreadHandler.post(command);
    }
}

private AppExecutors(Executor diskIO, Executor mainThread,
Executor networkIO)
{
    this.diskIO = diskIO;
    this.mainThread = mainThread;
    this.networkIO = networkIO;
}

public static AppExecutors getInstance()
{
    if (sInstance == null)
    {
        synchronized (LOCK)
        {
```

```

        sInstance = new AppExecutors(
            Executors.newSingleThreadExecutor(),
            Executors.newFixedThreadPool(3),
            new MainThreadExecutor());
    }
}
return sInstance;
}

public Executor getDiskIO()
{
    return diskIO;
}

public Executor getMainThread()
{
    return mainThread;
}

public Executor getNetworkIO()
{
    return networkIO;
}
}

```

Se definen 5 parámetros:

- **LOCK:** el semáforo para controlar la sincronía, que se implementará igual que en la anterior clase.
- **sInstance:** representa la instancia de la propia clase, que se controlará que solo haya una en el programa siguiendo el patrón singleton.
- **diskIO:** que se utilizará para comandos de entrada y salida de disco.
- **mainThread:** representa el hilo principal.
- **networkIO:** representa entradas y salidas provenientes de la red.

En primer lugar se crea una clase que implementa la clase `Executor`. Su función es la de recibir comandos y ejecutarlos a través de un `Runnable`, que iniciará un hilo de ejecución paralelo. En el método `getInstance` se implementa el `Singleton` se crea el `AppExecutor`, en el que se crea un nuevo executor al que se fija como número de hilos máximos posibles 3.

Lo siguiente será crear un adaptador para que rellene el `ListView`. En el package `adapter` se crea la clase `UsuariosAdapter`:

```
package com.example.ejemplodialog.adapter;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;

import com.example.ejemplodialog.model.Usuario;

import java.util.ArrayList;
import java.util.List;

public class UsuariosAdapter extends ArrayAdapter<Usuario>
{
    private final Context context;
    private List<Usuario> mUsuarioList;

    public UsuariosAdapter(@NonNull Context context, int resource)
    {
        super(context, resource);
        this.context = context;
        this.mUsuarioList = new ArrayList<>();
    }

    @Override
    public int getCount()
    {
        return mUsuarioList.size();
    }

    @Nullable
    @Override
    public Usuario getItem(int position)
    {
        return mUsuarioList.get(position);
    }

    @NonNull
    @Override
    public View getView(int position, @Nullable View convertView,
        @NonNull ViewGroup parent)
    {
        LayoutInflater inflater = LayoutInflater.from(context);
        View view =
        inflater.inflate(android.R.layout.simple_list_item_1, parent,
        false);
```

```

        TextView tvUsuario = view.findViewById(android.R.id.text1);
        tvUsuario.setText(mUsuarioList.get(position).getUsuario());
        return view;
    }

    public void setUsuarios(List<Usuario> usuariosList)
    {
        mUsuarioList = usuariosList;
        notifyDataSetChanged();
    }

    public List<Usuario> getUsuarios()
    {
        return mUsuarioList;
    }
}

```

Se accede a la clase RegisterActivity.java y se programa lo siguiente:

```

package com.example.ejemplodialog;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.lifecycle.Observer;

import com.example.ejemplodialog.adapter.UsuariosAdapter;
import com.example.ejemplodialog.bbdd.AppDatabase;
import com.example.ejemplodialog.bbdd.AppExecutors;
import com.example.ejemplodialog.model.Usuario;

```

```
import java.util.List;

public class RegisterActivity extends AppCompatActivity
{
    private ListView lvUsers;
    private Button btnRegistrarNuevo;
    private EditText etNombre;
    private EditText etPassword;
    private EditText etRePassword;

    private AppDatabase mDb;
    private UsuariosAdapter usuariosAdapter;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_register);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) ->
        {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());

            v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom);

            return insets;

        });

        mDb =
AppDatabase.getInstance(getApplicationContext());
    }
}
```

```

        Toast.makeText(getApplicationContext(), "Base de datos
        preparada", Toast.LENGTH_SHORT).show();

        lvUsers = findViewById(R.id.lvUsers);

        usuariosAdapter = new
        UsuariosAdapter(getApplicationContext(), 1);

        lvUsers.setAdapter(usuariosAdapter);

        etNombre = findViewById(R.id.etRegistroUser);
        etPassword = findViewById(R.id.etRegistroPassword);
        etRePassword =
        findViewById(R.id.etRegistroRePassword);

        btnRegistrarNuevo =
        findViewById(R.id.btnNuevoUsuario);
        btnRegistrarNuevo.setOnClickListener(new
        View.OnClickListener()
        {
            @Override
            public void onClick(View v)
            {
                String user = etNombre.getText().toString();

                String password =
                etPassword.getText().toString();

                guardar(user, password);

            }
        });

        consultar();
    }

    private void guardar(String user, String password)
    {
        final Usuario usuario = new Usuario(user, password);
    }

```



```

        AppExecutors.getInstance().getDiskIO().execute(new
Runnable()
{
    @Override
    public void run()
    {
        mDb.usuariosDao().insertUsuario(usuario);
    }
});
}

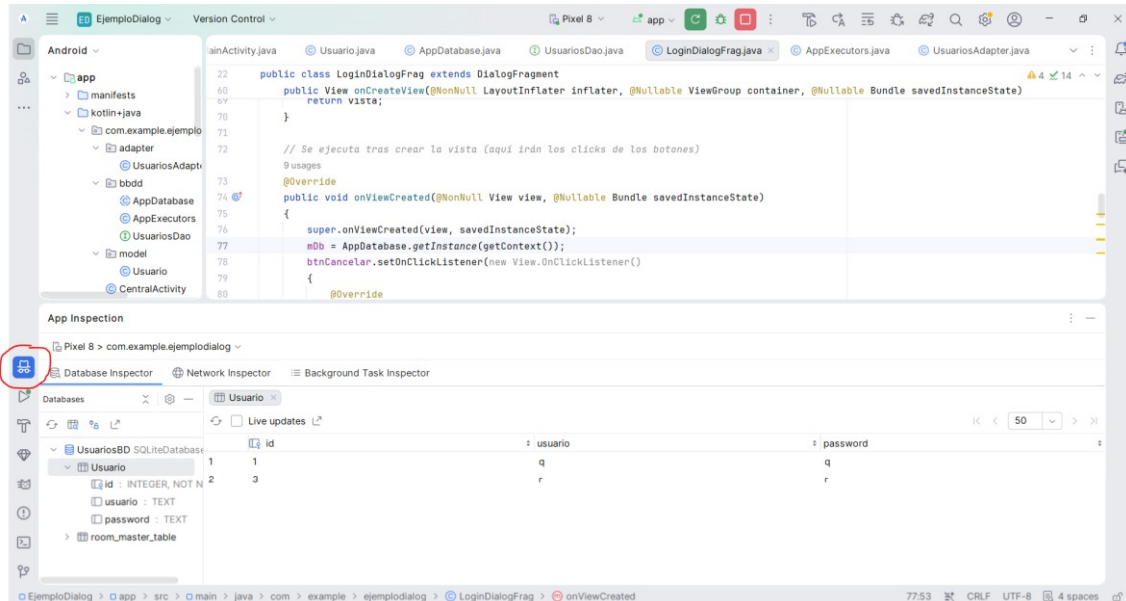
private void consultar()
{
    mDb.usuariosDao().loadAllUsuarios().observe(this, new
Observer<List<Usuario>>()
{
    @Override
    public void onChanged(@Nullable List<Usuario>
usuarios)
    {
        usuariosAdapter.setUsuarios(usuarios);
    }
});
}
}

```

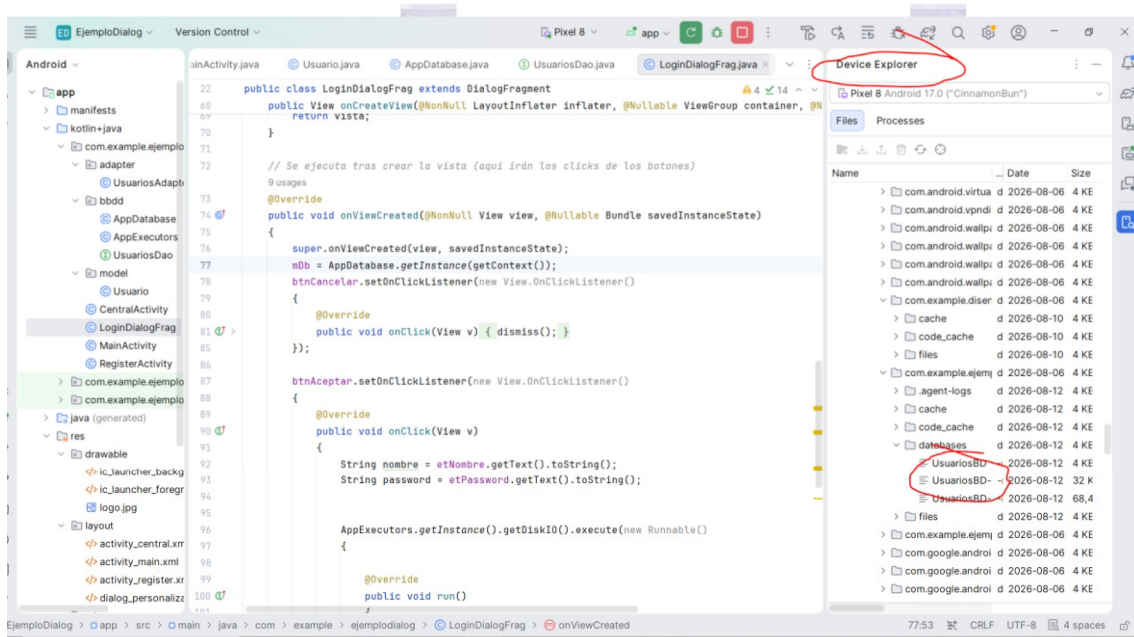
Se instancia la base de datos y el adaptador (el segundo parámetro se pasa a 1 porque no se ha creado un `LinearLayout` como elemento contenedor). Los datos se cargan llamando a `consultar()` en `onCreate`. La función observa el `LiveData` de `UsuariosDao`: como es una lista observable, basta con observarla una vez y cualquier cambio en la base de datos se refleja de inmediato en la vista, sin necesidad de recargar manualmente.

Al hacer click en nuevo se capturan los textos del formulario y se llama a la función guardar. Dentro de la misma se llama a la clase `AppExecutors` y se le pide una entrada/salida de disco a la que se le pasa un `Runnable`. De este modo el alta se realiza en un hilo paralelo y no se deja la interfaz esperando a que se termine.

Si se accede a View -> Tool Window -> App Inspection a la pestaña Database Inspector puede verse la base de datos generada:



El archivo físico puede verse desde View -> Tool Windows -> Device Explorer en la ruta /data/data/com.example.ejemplodialog/databases/UsuariosBD:



Para implementar la edición y eliminación se modifica lo siguiente:

```
package com.example.ejemplodialog;

import android.content.DialogInterface;
import android.graphics.Color;
import android.os.Bundle;
```

```
import android.text.Editable;
import android.text.TextWatcher;
import android.view.View;
import android.widget.AdapterView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.lifecycle.Observer;

import com.example.ejemplodialog.adapter.UsuariosAdapter;
import com.example.ejemplodialog.bbdd.AppDatabase;
import com.example.ejemplodialog.bbdd.AppExecutors;
import com.example.ejemplodialog.model.Usuario;

import java.util.List;

public class RegisterActivity extends AppCompatActivity
{
    private ListView lvUsers;
    private Button btnRegistrarNuevo;
    private Button btnUpdateUsuario;
    private EditText etNombre;
    private EditText etPassword;
```

```
private EditText etRePassword;

private int identificador = -1;

private AppDatabase mDb;
private UsuariosAdapter usuariosAdapter;

@Override
protected void onCreate(Bundle savedInstanceState)
{
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_register);

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) ->
    {
        Insets systemBars =
        insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top,
        systemBars.right, systemBars.bottom);
        return insets;
    });

    mDb =
    AppDatabase.getInstance(getApplicationContext());

    Toast.makeText(getApplicationContext(), "Base de datos
    preparada", Toast.LENGTH_SHORT).show();

    lvUsers = findViewById(R.id.lvUsers);
    usuariosAdapter = new
    UsuariosAdapter(getApplicationContext(), 1);
    lvUsers.setAdapter(usuariosAdapter);
}
```

```

        etNombre = findViewById(R.id.etRegistroUser);

        etPassword = findViewById(R.id.etRegistroPassword);

        etRePassword =
findViewById(R.id.etRegistroRePassword);

        btnRegistrarNuevo =
findViewById(R.id.btnNuevoUsuario);

        btnRegistrarNuevo.setOnClickListener(new
View.OnClickListener()

        {

            @Override

            public void onClick(View v)

            {

                String user = etNombre.getText().toString();
                String password =
etPassword.getText().toString();

                guardar(user, password);

            }

        });

        // Al pulsar un usuario de la lista, se cargan sus
datos en el formulario

        lvUsers.setOnItemClickListener(new
AdapterView.OnItemClickListener()

        {

            @Override

            public void onItemClick(AdapterView<?> parent,
View view, int position, long id)

            {

                Usuario usuario =
usuariosAdapter.getItem(position);

                identificador = usuario.getId();

                etNombre.setText(usuario.getUsuario());

                etPassword.setText(usuario.getPassword());

                etRePassword.setText(usuario.getPassword());

```

```

    }

    });

    // Mantener pulsado un usuario ofrece eliminarlo (con
confirmación)

    lvUsers.setOnItemClickListener(new
AdapterView.OnItemClickListener()

    {

        @Override

        public boolean onItemClick(AdapterView<?>
parent, View view, int position, long id)

        {

            final int idEliminar =
usuariosAdapter.getItem(position).getId();

            AlertDialog.Builder alerta = new
AlertDialog.Builder(RegisterActivity.this);

            alerta.setTitle("Advertencia");

            alerta.setMessage("¿Estás seguro de que
quieres eliminar el usuario?");

            alerta.setPositiveButton("Sí", new
DialogInterface.OnClickListener()

            {

                @Override

                public void onClick(DialogInterface
dialog, int which)

                {

                    eliminar(idEliminar);

                }

            });

            alerta.setNegativeButton("No", new
DialogInterface.OnClickListener()

            {

                @Override

                public void onClick(DialogInterface
dialog, int which)

```



```

        {

Toast.makeText(getApplicationContext(), "No se ha eliminado",
Toast.LENGTH_SHORT).show();

        }

    });

    alerta.show();

    return true;

    }

    });

    btnUpdateUsuario =
findViewById(R.id.btnUpdateUsuario);

    btnUpdateUsuario.setOnClickListener(new
View.OnClickListener()
    {

        @Override

        public void onClick(View v)
        {

            String user = etNombre.getText().toString();

            String password =
etPassword.getText().toString();

            actualizar(identificador, user, password);

        }

    });

    // Validación: la repetición de la contraseña debe
coincidir

    etRePassword.addTextChangedListener(new TextWatcher()
    {

        @Override

        public void beforeTextChanged(CharSequence s, int
start, int count, int after) { }
    }

```

```

@Override
    public void onTextChanged(CharSequence s, int
start, int before, int count) { }

@Override
    public void afterTextChanged(Editable s)
    {
        String password =
etPassword.getText().toString();
        String repassword =
etRePassword.getText().toString();
        if (!password.equals(repassword))
        {
            btnRegistrarNuevo.setEnabled(false);
etRePassword.setBackgroundColor(Color.RED);
        }
        else
        {
            btnRegistrarNuevo.setEnabled(true);
etRePassword.setBackgroundColor(Color.WHITE);
        }
    }

consultar();
}

private void guardar(String user, String password)
{
    final Usuario usuario = new Usuario(user, password);
    AppExecutors.getInstance().getDiskIO().execute(new
Runnable()

```

```

    {

        @Override

        public void run()

        {

            mDb.usuariosDao().insertUsuario(usuario);

        }

    });

}

private void actualizar(final int id, final String user,
final String password)

{

    AppExecutors.getInstance().getDiskIO().execute(new
Runnable()

{

    @Override

    public void run()

    {

        Usuario usu =
mDb.usuariosDao().loadUsuarioById(id);

        if (usu != null)

        {

            usu.setUsuario(user);

            usu.setPassword(password);

            mDb.usuariosDao().updateUsuario(usu);

        }

    }

});

}

private void eliminar(final int id)

{

```

```

        AppExecutors.getInstance().getDiskIO().execute(new
Runnable()
        {
            @Override
            public void run()
            {
                Usuario usu =
mDb.usuariosDao().loadUsuarioById(id);
                if (usu != null)
                {
                    mDb.usuariosDao().delete(usu);
                }
            }
        });
    }

    private void consultar()
    {
        mDb.usuariosDao().loadAllUsuarios().observe(this, new
Observer<List<Usuario>>()
        {
            @Override
            public void onChanged(@Nullable List<Usuario>
usuarios)
            {
                usuariosAdapter.setUsuarios(usuarios);
            }
        });
    }
}

```

Se implementa el inicio de sesión contra la base de datos. Para ello en UsuariosDao se crea una nueva consulta que preguntará por el usuario y contraseña introducidos:

```
package com.example.ejemplodialog.bbdd;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.Query;
import androidx.room.Update;

import com.example.ejemplodialog.model.Usuario;

import java.util.List;

@Dao
public interface UsuariosDao
{
    @Query("SELECT * FROM Usuario ORDER BY id")
    LiveData<List<Usuario>> loadAllUsuarios();

    @Insert
    void insertUsuario(Usuario usuario);

    @Update
    void updateUsuario(Usuario usuario);

    @Delete
    void delete(Usuario usuario);

    @Query("SELECT * FROM Usuario WHERE id = :id")
    Usuario loadUsuarioById(int id);
}
```

```
@Query("SELECT * FROM Usuario WHERE usuario = :usu AND
password = :pass")

Usuario loadUsuarioByNamePass(String usu, String pass);

}
```

En la clase LoginDialogFrag se implementa el acceso a la base de datos:

```
package com.example.ejemplodialog;

import android.app.Dialog;
import android.content.DialogInterface;
import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.DialogFragment;

import com.example.ejemplodialog.bbdd.AppDatabase;
import com.example.ejemplodialog.bbdd.AppExecutors;
import com.example.ejemplodialog.model.Usuario;

public class LoginDialogFrag extends DialogFragment
{
    private EditText etNombre;
    private EditText etPassword;
    private Button btnAceptar;
```



```
private Button btnCancelar;

private AppDatabase mDb;

public LoginDialogFrag()
{
    super();
}

@Override
public void onCancel(@NonNull DialogInterface dialog)
{
    super.onCancel(dialog);
}

@Override
public void onDetach()
{
    super.onDetach();
}

@NonNull
@Override
public Dialog onCreateDialog(@Nullable Bundle savedInstanceState)
{
    Dialog dialog =
super.onCreateDialog(savedInstanceState);

    dialog.setTitle("Login");

    return dialog;
}

@Nullable
@Override
```

```

    public View onCreateView(@NonNull LayoutInflater inflater,
        @Nullable ViewGroup container, @Nullable Bundle
        savedInstanceState)
    {
        View vista =
inflater.inflate(R.layout.dialog_personalizado, container,
false);

        etNombre = vista.findViewById(R.id.etUser);
        etPassword = vista.findViewById(R.id.etPassword);
        btnAceptar = vista.findViewById(R.id.btnAceptar);
        btnCancelar = vista.findViewById(R.id.btnCancelar);

        return vista;
    }

    @Override
    public void onViewCreated(@NonNull View view, @Nullable
Bundle savedInstanceState)
    {
        super.onViewCreated(view, savedInstanceState);

        mDb = AppDatabase.getInstance(getContext());

        btnCancelar.setOnClickListener(new
View.OnClickListener()
        {
            @Override
            public void onClick(View v)
            {
                dismiss();
            }
        });
    }
}

```

```

        btnAceptar.setOnClickListener(new
View.OnClickListener()

    {

        @Override

        public void onClick(View v)

        {

            final String nombre =
etNombre.getText().toString();

            final String password =
etPassword.getText().toString();

AppExecutors.getInstance().getDiskIO().execute(new Runnable()

    {

        @Override

        public void run()

        {

            // Consulta en segundo plano

            final Usuario usu =
mDb.usuariosDao().loadUsuarioByNamePass(nombre, password);

            // Vuelta al hilo principal para tocar
la interfaz

AppExecutors.getInstance().getMainThread().execute(new
Runnable()

    {

        @Override

        public void run()

        {

            if (usu != null)

            {

                Intent intent = new
Intent(getContext(), CentralActivity.class);

                startActivity(intent);

```

```
}  
else  
{  
  
Toast.makeText(getApplicationContext(), "Usuario o contraseña  
incorrectos", Toast.LENGTH_SHORT).show();  
  
dismiss();  
}  
}  
});  
}  
});  
}  
});  
}
```

Centro de Estudio

El acceso a la base de datos se realiza contra la tabla de usuarios mediante la consulta `loadUsuarioByNamePass`, que busca un usuario cuyo nombre y contraseña coincidan con los introducidos. Si la consulta devuelve un usuario, las credenciales son correctas y se accede a `CentralActivity`; en caso contrario se muestra un aviso y se cierra el diálogo.

Es importante tener en cuenta la regla de concurrencia de Android aplicada a Room: **las operaciones sobre la base de datos no pueden ejecutarse en el hilo principal**, ya que lo bloquearían y la aplicación se congelaría. Por ello, la consulta se lanza en un hilo secundario a través de `AppExecutors.getInstance().getDiskIO().execute(...)`, cuyo `Runnable` contiene el código que se ejecuta en segundo plano.

Ahora bien, existe la regla complementaria: **solo el hilo principal puede modificar la interfaz**. Como abrir una actividad (`startActivity`), mostrar un `Toast` o cerrar el diálogo (`dismiss`) son acciones sobre la interfaz, no pueden hacerse desde el hilo secundario. Por eso, una vez obtenido el resultado de la consulta, se vuelve al hilo principal mediante `AppExecutors.getInstance().getMainThread().execute(...)`, y es dentro de ese segundo `Runnable` donde se decide, según el resultado, si se abre `CentralActivity` o se avisa del error.

En resumen, el patrón es siempre el mismo: el trabajo pesado (la consulta a la base de datos) se realiza en segundo plano con `getDiskIO`, y la actualización de la interfaz que dependa de ese trabajo se devuelve al hilo principal con `getMainThread`.